From Human Speech Transcripts to Conservative Android Function Calls: A Leakage-Audited Evaluation of a Small Language Model

[Removed for double-blind review]

[1][Removed for double-blind review]

Abstract. *Voice interfaces for mobile devices must distinguish a supported command from an unsupported or ambiguous request before any Android API is invoked. This paper presents a reproducible, deliberately narrow evaluation of a Qwen3.5-2B language model as a text-to-function router for two Android-aligned operations: media play/pause and volume up/down. To obtain human-origin English command text without claiming that an existing corpus is an Android corpus, we derive an auditable benchmark from Fluent Speech Commands (FSC). Only four native FSC semantic combinations are mapped to calls; all other source examples are retained as explicit policy-abstention negatives. We compare zero-shot prompting with a LoRA adapter on the official speaker-disjoint split and on a stricter phrase-disjoint split that prevents normalized templates from crossing train, development, and test. The experiment consumes transcriptions, not waveforms, and therefore does not evaluate automatic speech recognition, physical Android execution, or on-device inference. On the official test, the zero-shot model reaches 39.97% exact match and the adapted model reaches 100%; because 244–247 normalized templates overlap across the official split pairs, this result is treated as a leakage warning rather than broad evidence of robustness. On the phrase-disjoint test, zero-shot reaches 40.33% exact match and LoRA reaches 100%. The contribution is a measurement protocol and an auditable baseline, not a claim of general Android command understanding.*

Keywords: function calling; Android; speech commands; small language models; abstention; data leakage.

1. Motivation and Scope

The original project goal is a specialized small language model for Android commands in Brazilian Portuguese. The available human command corpus, however, is not a native Android or Brazilian-Portuguese corpus. This phase therefore uses English human speech data as a temporary benchmark and makes the resulting scope explicit. The historical Portuguese synthetic data in the repository is retained as a software harness, not presented as human evidence.

The operational question is narrower than whether a language model can control Android. We ask whether a small model can transform a human-origin command transcription into a validated canonical object, or abstain when the source semantics do not belong to the evaluated contract. No generated object is executed. Android API names identify an intended integration boundary only.

The phase addresses four research questions:

1. Can Qwen3.5-2B produce valid canonical JSON for a two-tool Android-aligned contract?
2. What improvement does a LoRA adapter provide over the same model and prompt without adaptation?
3. How does performance change when normalized phrase templates are held out, rather than only speakers?
4. Does the abstention policy prevent unsupported FSC semantics from being silently converted into calls?

2. Related Work

Small language models are increasingly used as structured routers around deterministic software components. This setting differs from open-ended dialogue: a syntactically valid but semantically wrong call can be more harmful than a refusal. Parameter-efficient adaptation with LoRA [1] provides a practical way to specialize a local checkpoint while preserving a reproducible base model.

Fluent Speech Commands is a human-recorded English speech-command corpus introduced for spoken-language understanding [2]. Its native intents concern a smart-home assistant rather than Android APIs. Consequently, a direct use of its labels as Android gold would conflate source annotation with an experimenter's semantic interpretation. Our protocol keeps the source labels and records every bridge rule explicitly.

The data-splitting issue is central in command corpora: multiple speakers may repeat the same short template. An official speaker-disjoint split evaluates speaker transfer, but it does not necessarily evaluate lexical generalization. We therefore add a template-disjoint audit and report the two protocols separately.

3. Methodology

3.1. Corpus and Conservative Semantic Bridge

We use Fluent Speech Commands (FSC), containing 30,043 human recordings, 97 speakers, and 31 native intents. The source provides official train, validation, and test files with speaker separation. The archive and source CSV hashes are recorded in the project manifest; the raw data is kept outside the Git repository.

The corpus is not an Android command dataset. We define a two-tool registry and a deterministic bridge that accepts only four native semantic combinations, shown in Table 1.

Table 1. Only four native FSC combinations are mapped to calls.

Native action	Native object	Derived target
activate	music	`media_control(action=play)`
deactivate	music	`media_control(action=pause)`
increase	volume	`volume_adjust(direction=up)`
decrease	volume	`volume_adjust(direction=down)`

Every other FSC example receives the policy target `abstain`, with a null tool and empty arguments. This is a derived policy label, not a native FSC gold annotation.

The benchmark balances supported calls and unsupported policy negatives independently within each split, avoiding domination by the larger set of unrelated smart-home intents.

The registry is intentionally narrow. `media_control` represents the Android `MediaSessionManager` boundary and accepts `play` or `pause`; `volume_adjust` represents the `AudioManager.adjustStreamVolume` boundary and accepts `up` or `down`. The experiment does not request permissions, call either API, or claim that the model is safe to deploy without a validator and policy layer.

3.2. Leakage-Audited Splits

We preserve the official speaker-disjoint split and construct a phrase-disjoint split. A template is the Unicode-normalized, case-folded, whitespace-normalized, punctuation-stripped transcription; its stable SHA-256 prefix is stored as `template_id`. A template group is assigned to only one split within its native FSC label. Speakers may occur in several splits in the phrase-disjoint protocol by design.

Table 2. Balanced derived datasets and leakage controls.

Protocol	Train	Dev	Test	Total	Control
Official speaker-disjoint	11,442	1,580	1,934	14,956	speakers
Phrase-disjoint	10,458	2,202	2,296	14,956	templates

Both datasets contain 7,478 call examples and 7,478 abstention examples. The official test contains 967 calls and 967 policy negatives. The phrase-disjoint test contains 1,148 calls and 1,148 policy negatives. Dataset validation checks schema, target validity, duplicate IDs, expected locale, split assignment, and the selected group-disjointness invariant. The official split has no speaker intersection but has 244–247 template intersections across split pairs; the phrase-disjoint split has no template intersection.

3.3. Model and Training Protocol

The base checkpoint is Qwen3.5-2B loaded from the local Hugging Face cache. The model receives an English system instruction, the compact two-tool catalog, and one transcription. It must emit exactly `action`, `tool`, and `arguments` as JSON. The baseline uses the canonical prompt without adapter weights.

The adapted model uses LoRA with rank 16, alpha 32, dropout 0.05, and all-linear target modules. Training uses two epochs, batch size 4, gradient accumulation 4, learning rate 2×10^{-4}, maximum sequence length 1,024, bf16 autocast, and seed 20260830. The official run used 11,442 training records and 1,432 optimizer steps per epoch on an NVIDIA RTX 5090 with 32 GB VRAM. The phrase-disjoint run uses the same hyperparameters and hardware, changing only the split protocol.

The model is evaluated as a server-side text router. There is no microphone input, ASR stage, Android APK execution, permission flow, battery test, thermal test, or physical-device latency result in this phase.

3.4. Metrics

For every test item we report JSON validity, canonical validity, exact match of the canonical target, action accuracy, tool-selection accuracy on call gold items, argument exactness

overall and conditional on the selected tool, and abstention precision, recall, and F1. The evaluator also reports missing/extra predictions, invalid-output samples, and the same measurements grouped by explicit mapping rule. Wilson 95% intervals are reported for proportions; abstention F1 uses deterministic bootstrap percentile intervals with 2,000 resamples and seed 20260830. Latency is recorded per generated request as an engineering observation, not as an on-device claim.

4. Results

4.1. Official Speaker-Disjoint Test

The zero-shot baseline produced 1,934 predictions. It achieved 100% JSON parseability but only 68.77% canonical validity and 39.97% exact match. Abstention F1 was 61.08%. Call selection was uneven: exact match was 57.85% for play, 31.43% for pause, and 0% for both volume directions. On unsupported policy negatives, abstention recall was 69.29%.

The LoRA adapter produced 1,934 valid predictions and achieved 100% on every reported point estimate in this test: exact match, canonical validity, action, tool, arguments, and abstention F1. The Wilson lower bound for exact match is 99.80% with 1,934 observations.

Table 3. Official speaker-disjoint test on 1,934 examples.

Metric	Zero-shot	LoRA	LoRA lower bound
JSON valid	100.00%	100.00%	99.80%
Canonical valid	68.77%	100.00%	99.80%
Exact match	39.97%	100.00%	99.80%
Action accuracy	39.97%	100.00%	99.80%
Tool selection (calls)	10.65%	100.00%	99.60%
Argument exact (calls)	10.65%	100.00%	99.60%
Abstention F1	61.08%	100.00%	100.00%

This apparent perfect result is not sufficient evidence of broad generalization. The official split is speaker-disjoint but has 244–247 normalized templates in common across split pairs. The adapted model can exploit lexical repetition while learning the mapping. This is the motivation for the phrase-disjoint run.

The recorded GPU generation latency was 361.7 ms on average for the adapted model (median 359.0 ms, 95th percentile 396.6 ms) and 312.5 ms on average for the baseline. These are server measurements for single-request text generation and must not be interpreted as smartphone performance.

4.2. Phrase-Disjoint Test

The phrase-disjoint adapter was trained and evaluated with the same code and hyperparameters. On 2,296 test items, the zero-shot baseline reached 74.09% canonical validity, 40.33% exact match, tool selection of 10.63%, and abstention F1 of 58.97%. The LoRA adapter reached 100% on all point estimates: JSON validity, canonical validity, exact

match, action, tool, arguments, and abstention F1. The Wilson lower bound for exact match is 99.83%.

The per-rule pattern is informative. Zero-shot exact match was 100% for play, 0% for pause, 0% for decrease-volume, and 0% for increase-volume; abstention recall on derived negatives was 70.03%. LoRA reached 100% on all four mapped rules and on the derived abstention policy.

The two split protocols therefore agree on the narrow task, but they do not establish Android command understanding beyond the four manually declared semantic bridges. The absence of template overlap removes one important leakage channel; it does not replace an independently annotated Android test.

5. Discussion

The experiment separates three questions that are often conflated. First, a human-origin speech corpus can provide command text, but it does not become an Android corpus merely because a researcher writes a mapping. Second, a model can learn a strict JSON contract while still failing semantic routing, as the zero-shot volume results show. Third, a high score on an official speaker split can reflect repeated lexical templates rather than robustness to new wording.

The abstention target is useful as a safety-oriented interface, but it is also the most assumption-sensitive part of the benchmark. Unsupported FSC examples are not human-annotated Android refusals; they are policy negatives created by the experimenter. The correct claim is therefore "abstention under a declared derived policy," not "safe refusal for Android."

The adapter's official-split perfect score should be treated as an overfit signal until compared with the phrase-disjoint protocol and, ultimately, with a held-out human evaluation designed for Android commands. Even after lexical control, text-only evaluation leaves the ASR error channel unmeasured. A production pipeline needs an ASR model, confidence thresholding, schema validation, permission checks, and a policy gate between generated JSON and Android APIs.

6. Reproducibility and Data Governance

The repository records the mapping contract, generator, source-file hashes, split summaries, validator, training script, baseline script, evaluator, tests, and aggregate metrics. The raw FSC archive and derived JSONL files remain server-local. Because the source is distributed under CC BY-NC-ND 4.0 for academic research, derived transcripts/labels and per-example predictions are ignored by Git and are not redistributed with the repository. The manifest preserves provenance without shipping restricted data.

All phase-1 tests pass in the Neuromancer environment. Training and evaluation use the RTX 5090; the Android phone is not required for this phase.

7. Limitations and Next Gates

1. The only human corpus used in this phase is English FSC; no Brazilian-Portuguese human command set is claimed.

2. FSC is not an Android corpus; four mappings are manually specified and all other targets are derived abstentions.
3. The input is transcription text, not audio; ASR robustness is unmeasured.
4. The official result is vulnerable to lexical repetition; the phrase-disjoint control was executed, but an independent Android test is still required before publication.
5. The contract covers only play/pause and volume up/down.
6. No Android API is called, no permission is exercised, and no safety guarantee is established.
7. No on-device or physical-phone result is reported. The Qwen3.5-2B experiment runs on the RTX 5090.
8. The source license requires an explicit redistribution audit before any public data release.

The next critical gates are to add an independently annotated Android-command test, insert an ASR stage, benchmark the validator and policy layer, and only then connect the phone for physical execution and on-device measurements.

8. Conclusion

This phase establishes an auditable benchmark and an honest experimental boundary for the first article. The model reaches 100% on both declared split protocols, while the zero-shot baseline remains near 40% exact match; this is evidence that the model learns the narrow declared contract, not evidence of general Android command understanding. The article should still include an independent human/Android evaluation or be framed explicitly as a protocol and preliminary engineering report.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[2] S. Lugosch et al. How to train your own neural network for keyword spotting. In *Interspeech*, 2019.

[3] Google. Android developer documentation: Media sessions and audio management. Technical documentation, accessed 2026.